

TP 4 : Analyse en temps quasi réel des mesures de capteurs

avec PySpark Structured Streaming et HDFS

1. Titre du TP

Traitement de flux de mesures de capteurs avec **PySpark Structured Streaming** à partir de fichiers déposés régulièrement dans **HDFS**.

2. Problématique

Dans un système de supervision, des fichiers contenant les mesures de plusieurs capteurs sont déposés régulièrement dans un dossier HDFS.

L'entreprise souhaite exploiter ces données dès leur arrivée afin de :

- surveiller l'évolution des mesures ;
- calculer des indicateurs en continu ;
- détecter rapidement les valeurs anormales ;
- afficher les résultats en temps quasi réel.

L'objectif est donc de mettre en place une application **PySpark Structured Streaming** capable de lire automatiquement les nouveaux fichiers CSV déposés dans HDFS et de traiter ces données en flux quasi temps réel.

3. Objectifs pédagogiques

À la fin de ce TP, l'étudiant doit être capable de :

- comprendre le principe de Spark Structured Streaming ;
- utiliser HDFS comme source de streaming ;
- développer une application streaming avec PySpark ;
- définir un schéma explicite pour lire des fichiers CSV ;
- réaliser des traitements et des agrégations sur un flux ;
- configurer un checkpoint ;
- détecter des valeurs anormales ;
- exécuter une application PySpark avec `spark-submit`.

4. Prérequis

Avant de commencer, il faut disposer de :

- un cluster HDFS fonctionnel ;
- un cluster Spark Standalone fonctionnel ;
- Docker si l'environnement est conteneurisé ;
- Python installé ;
- PySpark installé ou disponible dans le conteneur Spark ;
- un répertoire HDFS pour les fichiers source ;
- un répertoire HDFS pour les checkpoints.

5. Données à traiter

Les fichiers CSV auront la structure suivante :

```
id,timestamp,capteur,valeur,unite
1,2026-03-30 09:00:00,CAPTEUR_TEMP_1,22.5,C
2,2026-03-30 09:00:05,CAPTEUR_TEMP_2,24.1,C
3,2026-03-30 09:00:10,CAPTEUR_HUM_1,60.3,%
4,2026-03-30 09:00:15,CAPTEUR_TEMP_1,23.2,C
```

5.1. Description des colonnes

Colonne	Description
id	Identifiant de la mesure
timestamp	Date et heure de la mesure
capteur	Nom du capteur
valeur	Valeur mesurée
unite	Unité de mesure

6. Travail demandé

L'étudiant doit réaliser les tâches suivantes :

1. Créer dans HDFS un dossier qui servira de source de streaming.
2. Créer un dossier HDFS pour les checkpoints.
3. Préparer plusieurs fichiers CSV contenant des mesures de capteurs.
4. Développer une application PySpark Structured Streaming.
5. Lire les nouveaux fichiers CSV depuis HDFS.
6. Utiliser un schéma explicite.
7. Calculer en streaming :
 - la moyenne des valeurs par capteur ;
 - la valeur minimale par capteur ;
 - la valeur maximale par capteur ;
 - le nombre de mesures par capteur.
8. Identifier les mesures anormales dépassant un seuil donné.
9. Exécuter l'application.
10. Déposer progressivement les fichiers dans HDFS.
11. Observer les résultats générés après chaque nouveau fichier.

7. Préparation de HDFS

Créer les dossiers nécessaires :

```
hdfs dfs -mkdir -p /streaming/capteurs
hdfs dfs -mkdir -p /streaming/checkpoints/capteurs_stats
hdfs dfs -mkdir -p /streaming/checkpoints/capteurs_alertes
```

Vérifier la création des dossiers :

```
hdfs dfs -ls /streaming
hdfs dfs -ls /streaming/checkpoints
```

Si vous souhaitez relancer le TP depuis le début, vous pouvez supprimer les anciens fichiers :

```
hdfs dfs -rm -r /streaming/capteurs/*
hdfs dfs -rm -r /streaming/checkpoints/capteurs_stats/*
hdfs dfs -rm -r /streaming/checkpoints/capteurs_alertes/*
```

8. Structure du projet

Créer la structure suivante :

```
tp-pyspark-streaming-capteurs/
    app.py
    data/
        capteurs_1.csv
        capteurs_2.csv
        capteurs_3.csv
```

9. Préparation des fichiers CSV

9.1. Fichier capteurs_1.csv

```
id,timestamp,capteur,valeur,unite
1,2026-03-30 09:00:00,CAPTEUR_TEMP_1,22.5,C
2,2026-03-30 09:00:05,CAPTEUR_TEMP_2,24.1,C
3,2026-03-30 09:00:10,CAPTEUR_HUM_1,60.3,%
4,2026-03-30 09:00:15,CAPTEUR_TEMP_1,23.2,C
```

9.2. Fichier capteurs_2.csv

```
id,timestamp,capteur,valeur,unite
5,2026-03-30 09:01:00,CAPTEUR_TEMP_1,25.4,C
6,2026-03-30 09:01:05,CAPTEUR_TEMP_2,26.8,C
7,2026-03-30 09:01:10,CAPTEUR_HUM_1,65.1,%
8,2026-03-30 09:01:15,CAPTEUR_TEMP_1,40.7,C
```

9.3. Fichier capteurs_3.csv

```
id,timestamp,capteur,valeur,unite
9,2026-03-30 09:02:00,CAPTEUR_TEMP_2,27.2,C
10,2026-03-30 09:02:05,CAPTEUR_HUM_1,70.6,%
11,2026-03-30 09:02:10,CAPTEUR_TEMP_1,21.9,C
12,2026-03-30 09:02:15,CAPTEUR_TEMP_2,45.3,C
```

Dans cet exemple, les valeurs 40.7 et 45.3 peuvent être considérées comme anormales si le seuil est fixé à 35.

10. Code complet de l'application PySpark

Créer un fichier nommé `app.py` :

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, min, max, count
from pyspark.sql.types import StructType, StructField
from pyspark.sql.types import IntegerType, StringType, DoubleType,
    TimestampType

# Creation de la session Spark
spark = SparkSession.builder \
    .appName("TP PySpark Structured Streaming - Capteurs HDFS") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")

# Definition du schema explicite des fichiers CSV
schema_capteurs = StructType([
    StructField("id", IntegerType(), True),
    StructField("timestamp", TimestampType(), True),
    StructField("capteur", StringType(), True),
    StructField("valeur", DoubleType(), True),
    StructField("unite", StringType(), True)
])

# Chemins HDFS
source_path = "hdfs://namenode:8020/streaming/capteurs"
checkpoint_stats = "hdfs://namenode:8020/streaming/checkpoints/
    capteurs_stats"
checkpoint_alertes = "hdfs://namenode:8020/streaming/checkpoints/
    capteurs_alertes"

# Lecture du flux depuis HDFS
df_stream = spark.readStream \
    .option("header", "true") \
    .option("maxFilesPerTrigger", 1) \
    .schema(schema_capteurs) \
    .csv(source_path)

# Affichage du schema
df_stream.printSchema()

# Calcul des statistiques par capteur
stats_capteurs = df_stream.groupBy("capteur") \
    .agg(
        avg("valeur").alias("moyenne_valeur"),
        min("valeur").alias("valeur_min"),
        max("valeur").alias("valeur_max"),
        count("*").alias("nombre_mesures")
    )

# Detection des valeurs anormales
seuil_anomalie = 35.0
```

```

alertes = df_stream.filter(col("valeur") > seuil_anomalie) \
    .select(
        "id",
        "timestamp",
        "capteur",
        "valeur",
        "unite"
    )

# Ecriture des statistiques dans la console
query_stats = stats_capteurs.writeStream \
    .outputMode("complete") \
    .format("console") \
    .option("truncate", "false") \
    .option("checkpointLocation", checkpoint_stats) \
    .trigger(processingTime="10 seconds") \
    .start()

# Ecriture des alertes dans la console
query_alertes = alertes.writeStream \
    .outputMode("append") \
    .format("console") \
    .option("truncate", "false") \
    .option("checkpointLocation", checkpoint_alertes) \
    .trigger(processingTime="10 seconds") \
    .start()

# Attendre l'arrêt des traitements
spark.streams.awaitAnyTermination()

```

11. Explication du code

11.1. Création de la session Spark

```

spark = SparkSession.builder \
    .appName("TP PySpark Structured Streaming - Capteurs HDFS") \
    .getOrCreate()

```

Cette instruction crée une session Spark. Elle représente le point d'entrée principal pour utiliser PySpark.

11.2. Définition du schéma

```

schema_capteurs = StructType([
    StructField("id", IntegerType(), True),
    StructField("timestamp", TimestampType(), True),
    StructField("capteur", StringType(), True),
    StructField("valeur", DoubleType(), True),
    StructField("unite", StringType(), True)
])

```

En Structured Streaming, il est recommandé de définir un schéma explicite. Spark doit connaître la structure des fichiers avant de démarrer le flux.

11.3. Lecture du flux depuis HDFS

```
df_stream = spark.readStream \
    .option("header", "true") \
    .option("maxFilesPerTrigger", 1) \
    .schema(schema_capteurs) \
    .csv(source_path)
```

Cette partie permet à Spark de surveiller le dossier HDFS `/streaming/capteurs`. Chaque nouveau fichier CSV déposé dans ce dossier sera automatiquement traité.

L'option suivante permet de traiter un fichier à chaque micro-batch :

```
.option("maxFilesPerTrigger", 1)
```

Cela permet de mieux observer les résultats dans le cadre d'un TP.

11.4. Calcul des statistiques

```
stats_capteurs = df_stream.groupBy("capteur") \
    .agg(
        avg("valeur").alias("moyenne_valeur"),
        min("valeur").alias("valeur_min"),
        max("valeur").alias("valeur_max"),
        count("*").alias("nombre_mesures")
    )
```

Cette partie calcule les indicateurs suivants pour chaque capteur :

- la moyenne des valeurs ;
- la valeur minimale ;
- la valeur maximale ;
- le nombre total de mesures.

11.5. Détection des valeurs anormales

```
alertes = df_stream.filter(col("valeur") > seuil_anomalie)
```

Cette instruction sélectionne uniquement les mesures dont la valeur dépasse le seuil fixé.

Dans ce TP, le seuil est :

```
seuil_anomalie = 35.0
```

11.6. Modes de sortie

Pour les statistiques, on utilise le mode `complete` :

```
.outputMode("complete")
```

Ce mode affiche toute la table des résultats agrégés à chaque micro-batch.

Pour les alertes, on utilise le mode `append` :

```
.outputMode("append")
```

Ce mode affiche uniquement les nouvelles lignes détectées.

12. Exécution de l'application

Se placer dans le dossier du projet :

```
cd tp-pyspark-streaming-capteurs
```

Exécuter l'application avec Spark Standalone :

```
spark-submit \
  --master spark://spark-master:7077 \
  app.py
```

Selon l'environnement utilisé, il est aussi possible d'exécuter l'application en mode local :

```
spark-submit --master local[*] app.py
```

Ou bien avec un master Spark local :

```
spark-submit --master spark://localhost:7077 app.py
```

13. Dépôt progressif des fichiers dans HDFS

Dans un autre terminal, déposer le premier fichier :

```
hdfs dfs -put data/capteurs_1.csv /streaming/capteurs/
```

Observer les résultats dans la console Spark.

Déposer ensuite le deuxième fichier :

```
hdfs dfs -put data/capteurs_2.csv /streaming/capteurs/
```

Puis déposer le troisième fichier :

```
hdfs dfs -put data/capteurs_3.csv /streaming/capteurs/
```

Lister les fichiers déposés dans HDFS :

```
hdfs dfs -ls /streaming/capteurs
```

14. Exemple de résultat attendu

Après le dépôt de plusieurs fichiers, Spark peut afficher un résultat proche de celui-ci :

```
+-----+-----+-----+-----+-----+
|capteur      |moyenne_valeur|moyenne_valeur_min|moyenne_valeur_max|nombre_mesures|
+-----+-----+-----+-----+-----+
|CAPTEUR_TEMP_1|26.74         |21.9              |40.7              |5              |
|CAPTEUR_TEMP_2|30.85         |24.1              |45.3              |4              |
|CAPTEUR_HUM_1 |65.33         |60.3              |70.6              |3              |
+-----+-----+-----+-----+-----+
```

Les alertes peuvent apparaître comme ceci :

```
+---+-----+-----+-----+-----+
|id |timestamp           |capteur           |valeur|unite|
+---+-----+-----+-----+-----+
|8  |2026-03-30 09:01:15|CAPTEUR_TEMP_1   |40.7  |C    |
```

```
|12 |2026-03-30 09:02:15|CAPTEUR_TEMP_2|45.3 |C |
+---+-----+-----+-----+-----+-----+
```

15. Variante : enregistrer les alertes dans HDFS

Au lieu d'afficher les alertes uniquement dans la console, on peut les enregistrer dans HDFS au format Parquet.

Ajouter le chemin de sortie :

```
output_alertes = "hdfs://namenode:8020/streaming/output/alertes"
```

Puis remplacer l'écriture console des alertes par :

```
query_alertes = alertes.writeStream \
    .outputMode("append") \
    .format("parquet") \
    .option("path", output_alertes) \
    .option("checkpointLocation", checkpoint_alertes) \
    .trigger(processingTime="10 seconds") \
    .start()
```

Créer le dossier de sortie si nécessaire :

```
hdfs dfs -mkdir -p /streaming/output/alertes
```

Vérifier les fichiers générés :

```
hdfs dfs -ls /streaming/output/alertes
```

16. Questions de compréhension

1. Pourquoi doit-on définir un schéma explicite en Structured Streaming ?
2. Quel est le rôle du checkpoint ?
3. Quelle est la différence entre `append` et `complete` dans `writeStream` ?
4. Pourquoi utilise-t-on `maxFilesPerTrigger` ?
5. Que se passe-t-il si on redémarre l'application avec le même checkpoint ?
6. Pourquoi Spark Structured Streaming est-il considéré comme un traitement en micro-batch ?
7. Dans quel cas Kafka serait plus adapté que HDFS comme source de streaming ?
8. Comment modifier le programme pour enregistrer les statistiques dans HDFS au lieu de les afficher dans la console ?
9. Quelle est la différence entre un traitement batch classique et un traitement Structured Streaming ?
10. Pourquoi le mode `complete` est-il utilisé pour les agrégations ?

17. Travail complémentaire

Modifier l'application afin de :

- détecter les valeurs anormales selon le type de capteur ;
- utiliser un seuil différent pour les capteurs de température et d'humidité ;
- enregistrer les statistiques dans HDFS au format Parquet ;
- ajouter une colonne `statut` qui prend la valeur `NORMAL` ou `ANORMAL` ;
- afficher uniquement les capteurs dont la moyenne dépasse un seuil donné.

Exemple de règle possible :

- température anormale si la valeur dépasse 35 ;
- humidité anormale si la valeur dépasse 80.

18. Conclusion

Dans ce TP, nous avons développé une application **PySpark Structured Streaming** permettant de traiter des fichiers CSV déposés progressivement dans HDFS.

Ce TP a permis de comprendre :

- le fonctionnement de Spark Structured Streaming ;
- la lecture de fichiers en mode streaming ;
- l'utilisation d'un schéma explicite ;
- le calcul d'indicateurs en continu ;
- la détection de valeurs anormales ;
- l'importance des checkpoints ;
- l'exécution d'une application PySpark avec `spark-submit`.